SOF 103 Week 5: Arrays

> Generated by Review Agent on 2026-07-02 06:35 UTC
> Source: `SOF 103-Chapter 05 - Arrays2024_09.ppt`
> Classification confidence: 80%

---

1. Core Knowledge Outline

中文

* **数组是什么?**
* 一种数据结构，用于存储**相同类型**的多个元素。
* 所有元素在内存中**连续存放**。
* 通过**索引 (Index)** 访问，索引从 `0` 开始。
* **为什么使用数组?**
* 高效处理大量相关数据，避免声明无数个独立变量（如 `score1`, `score2`, ...）。
* **数组的声明与初始化**
* `类型 数组名[大小];` (例如: `int scores[5];`)
* 可以在声明时初始化: `int scores[5] = {90, 85, 92, 78, 88};`
* 可以省略大小，由编译器自动计算: `int scores[] = {90, 85, 92};`
* **访问数组元素**
* 使用下标运算符 `[]`: `scores[0]` (第一个元素), `scores[4]` (第五个元素)。
* 常见错误: **数组越界** (Index out of bounds)，访问不存在的索引。
* **数组与循环**
* 循环（特别是 `for` 循环）是处理数组的理想工具。
* 使用循环遍历数组，进行读取、修改、求和、查找等操作。
* **多维数组**
* 数组的数组，常用于表示表格或矩阵。
* 声明: `int matrix[3][4];` (3行4列)。
* 访问: `matrix[行索引][列索引]`。
* **数组与函数**

* 数组可以作为参数传递给函数。
* 传递的是数组的**首地址 (指针)**，而不是整个数组的副本（按引用传递）。
* 通常需要额外传递一个参数来表示数组的大小。

English
* **What is an Array?**
* A data structure that stores a **fixed-size sequential collection** of elements of the **same type**.
* Elements are stored in **contiguous memory locations**.
* Accessed via an **index**, starting from `0`.
* **Why Use Arrays?**
* To efficiently handle large amounts of related data without declaring many individual variables (e.g., `score1`, `score2`, ...).
* **Declaring and Initializing Arrays**
* `type arrayName[arraySize];` (e.g., `int scores[5];`)
* Initialize at declaration: `int scores[5] = {90, 85, 92, 78, 88};`
* Size can be omitted, letting the compiler infer it: `int scores[] = {90, 85, 92};`
* **Accessing Array Elements**
* Using the subscript operator `[]`: `scores[0]` (first element), `scores[4]` (fifth element).
* Common Pitfall: **Array Index Out of Bounds** – accessing an index that doesn't exist.
* **Arrays and Loops**
* Loops (especially `for` loops) are the perfect tool for working with arrays.
* Use loops to traverse, read, modify, sum, or search through array elements.
* **Multidimensional Arrays**
* An array of arrays, often used to represent tables or matrices.
* Declaration: `int matrix[3][4];` (3 rows, 4 columns).
* Access: `matrix[rowIndex][columnIndex]`.
* **Arrays and Functions**
* Arrays can be passed as arguments to functions.
* The **base address (pointer)** of the array is passed, not a copy of the entire

array (passed by reference).
* An additional parameter for the array's size is usually needed.

---

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
数组 (Array)	一种数据结构，用于存储固定数量的相同类型元素的集合。 / A data structure that stores a fixed-size collection of elements of the same type.	**一排有编号的邮箱。**每个邮箱（元素）大小相同，都有一个唯一的编号（索引）。 / **A row of numbered mailboxes**. Each mailbox (element) is the same size and has a unique number (index).
索引 (Index)	用于标识数组中特定元素位置的数字，通常从0开始。 / A number used to identify the position of a specific element in an array, typically starting from 0.	**公寓的房间号。**`101`房间是第一个，`102`是第二个。在C++中，房间号从`0`开始。 / **Apartment room numbers**. Room `101` is the first, `102` is the second. In C++, room numbers start from `0`.
数组越界 (Array Index Out of Bounds)	尝试访问索引小于0或大于等于数组大小的元素时发生的错误。 / An error that occurs when you try to access an element with an index less than 0 or greater than or equal to the array size.	**试图打开一个不存在的邮箱。**程序会崩溃或产生不可预测的结果。 / **Trying to open a mailbox that doesn't exist**. The program will crash or produce unpredictable results.
声明 (Declaration)	告诉编译器数组的名称、元素类型和大小。 / Telling the compiler the array's name, element type, and size.	**为你的邮箱群预订一排新的邮箱。**你需要告诉邮局有多少个邮箱以及它们的大小。 / **Reserving a new row of mailboxes for your community**. You need to tell the post office how many and what size.
初始化 (Initialization)	在声明数组的同时为其元素赋予初始值。 / Assigning initial values to the elements of an array at the time of its declaration.	**在你搬进新公寓时，把家具放进每个房间。 / Putting furniture into each room when you move into a new apartment**.
遍历 (Traversal)	顺序访问数组中的每个元素一次。 / Accessing each element	**邮递员沿着邮箱排，依次检查每个邮箱。 / A mail carrier walking**

Term	Definition	Memory Anchor / Analogy
	of an array in order, exactly once.	**down the row of mailboxes, checking each one in order**.
多维数组 (Multidimensional Array)	数组的数组，可以想象成一个表格或网格。 / An array of arrays, which can be visualized as a table or grid.	**一个Excel电子表格**。你需要行号和列号来定位一个特定的单元格。 / **An Excel spreadsheet**. You need a row number and a column number to find a specific cell.
按引用传递 (Pass by Reference)	当数组作为函数参数时，传递的是其内存地址，函数可以直接修改原始数组。 / When an array is passed to a function, its memory address is passed, allowing the function to directly modify the original array.	**你把你的公寓钥匙给了清洁工**。他们可以直接进入并打扫，而不需要你给他们一份公寓的复制品。 / **You give your apartment key to a cleaner**. They can enter and clean directly, instead of you giving them a copy of your apartment.

3. Critical Takeaways

中文

1. **索引从0开始**：这是初学者最容易犯的错误。`array[0]` 是第一个元素，`array[n-1]` 是最后一个元素。
2. **数组越界是万恶之源**：C++不会检查数组边界。访问越界元素会导致程序崩溃（段错误）或静默地损坏数据，产生难以调试的bug。
3. **数组名即指针**：当把数组传递给函数时，你实际上传递的是指向第一个元素的指针。这意味着函数内部对数组的修改会影响到原始数组。务必同时传递数组的大小。
4. **循环是数组的最佳拍档**：几乎所有的数组操作（输入、输出、求和、查找、排序）都离不开循环。`for` 循环是最常用的。
5. **区分声明和访问**：声明时 `int arr[10]` 的 `[10]` 表示大小。访问时 `arr[5]` 的 `[5]` 表示索引（第6个元素）。

English

1. **Indexing Starts at 0**: This is the most common beginner mistake. `array[0]` is the first element, and `array[n-1]` is the last.

2. **Array Bounds are Not Checked**: C++ does not check array boundaries. Accessing an out-of-bounds element can crash your program (segmentation fault) or silently corrupt data, leading to hard-to-find bugs.
3. **Array Name is a Pointer**: When you pass an array to a function, you are passing a pointer to its first element. This means modifications inside the function affect the original array. Always pass the array's size as well.
4. **Loops are an Array's Best Friend**: Almost all array operations (input, output, summing, searching, sorting) rely on loops. The `for` loop is the most common.
5. **Distinguish Declaration from Access**: In `int arr[10]`, the `[10]` in the declaration means size. In `arr[5]`, the `[5]` in an access means index (the 6th element).

---

4. Detailed Notes (AI-Expanded)

4.1 数组基础 / Array Basics

中文

想象一下，你需要存储班上50个学生的考试成绩。如果没有数组，你需要声明50个独立的变量：`score1`，`score2`，...，`score50`。这不仅繁琐，而且无法用循环高效地处理它们。

数组就是为了解决这个问题而生的。它是一个**容器**，可以存储**固定数量**的**相同类型**的元素。这些元素在计算机的内存中是**连续存放**的，就像一个并排的盒子队列。

声明一个数组，你需要指定三个东西：
1. **元素的类型**：例如 `int`，`double`，`char`。
2. **数组的名字**：遵循变量命名规则。
3. **数组的大小**：一个常量或常量表达式，用方括号 `[]` 括起来。

语法：`类型 数组名[大小];`

例如：`int studentScores[50];` 这行代码告诉编译器：“嘿，给我准备50个连续的 `int` 类型的内存空间，我给这整块空间取名叫 `studentScores`。”

访问数组元素，你需要使用数组名和元素的**索引**。索引从 `0` 开始，所以第一个元素是 `studentScores[0]`，最后一个元素是 `studentScores[49]`。

English

Imagine you need to store the test scores of 50 students in a class. Without arrays, you would need to declare 50 individual variables: `score1`, `score2`, ..., `score50`. This is not only tedious but also makes it impossible to process them efficiently with loops.

Arrays are designed to solve this problem. They are **containers** that can store a **fixed number** of elements of the **same type**. These elements are stored **contiguously** in the computer's memory, like a row of side-by-side boxes.

To **declare an array**, you need to specify three things:
1. **The type of elements**: e.g., `int`, `double`, `char`.
2. **The name of the array**: Follows variable naming rules.
3. **The size of the array**: A constant or constant expression, enclosed in square brackets `[]`.

Syntax: `type arrayName[size];`

For example: `int studentScores[50];` This line tells the compiler: "Hey, prepare 50 consecutive `int` memory spaces for me. I'm naming this whole block of space `studentScores`."

To **access an array element**, you use the array name and the element's **index**. The index starts from `0`, so the first element is `studentScores[0]`, and the last element is `studentScores[49]`.

4.2 数组的初始化 / Array Initialization

中文

你可以在声明数组的同时给它赋初值，这叫**初始化**。有多种方式：

1. **完全初始化**：`int scores[5] = {90, 85, 92, 78, 88};`
   花括号 `{}` 里的值按顺序赋给 `scores[0]` 到 `scores[4]`。

2. **部分初始化**：`int scores[5] = {90, 85};`
   只初始化前两个元素，剩余的元素（`scores[2]` 到 `scores[4]`）会被自动设置为 `0`。

3. **省略大小**：`int scores[] = {90, 85, 92, 78, 88};`
   编译器会自动根据花括号中值的个数来确定数组大小为5。这是一种非常方便且推荐的做法。

4. **全部初始化为0**：`int scores[5] = {0};`
   这是一种快速将所有元素设置为0的惯用法。

English

You can assign initial values to an array at the time of its declaration. This is called **initialization**. There are several ways:

1. **Full Initialization**: `int scores[5] = {90, 85, 92, 78, 88};`
   The values inside the curly braces `{}` are assigned to `scores[0]` through `scores[4]` in order.

2. **Partial Initialization**: `int scores[5] = {90, 85};`
   Only the first two elements are initialized. The remaining elements (`scores[2]` through `scores[4]`) are automatically set to `0`.

3. **Omitting the Size**: `int scores[] = {90, 85, 92, 78, 88};`
   The compiler automatically determines the array size (5) from the number of values in the braces. This is a very convenient and recommended practice.

4. **Initializing All to Zero**: `int scores[5] = {0};`
   This is a common idiom to quickly set all elements of an array to `0`.

4.3 数组与循环 / Arrays and Loops

中文

数组的真正威力在于与循环结合使用。`for` 循环是处理数组的标准工具。

示例：计算平均分

```
int scores[5] = {90, 85, 92, 78, 88};
int sum = 0;
int numStudents = 5;

for (int i = 0; i < numStudents; i++) {
    sum = sum + scores[i]; // 逐个累加每个学生的成绩
}

double average = (double)sum / numStudents;
```

在这个例子中，循环变量 `i` 从 `0` 到 `4`，完美地对应了数组的索引。`scores[i]` 让我们可以访问到每一个元素。

常见错误：忘记数组索引从0开始，或者循环条件写错（例如写成 `i ≤ numStudents`），都会导致**数组越界**。访问 `scores[5]` 是一个严重的错误，因为它访问了不属于数组的内存。

English

The real power of arrays is unleashed when combined with loops. The `for` loop is the standard tool for working with arrays.

Example: Calculating the Average Score

```
int scores[5] = {90, 85, 92, 78, 88};
int sum = 0;
int numStudents = 5;

for (int i = 0; i < numStudents; i++) {
    sum = sum + scores[i]; // Accumulate each student's score
}

double average = (double)sum / numStudents;
```

In this example, the loop variable `i` goes from `0` to `4`, perfectly matching the array's indices. `scores[i]` allows us to access each element.

Common Mistake: Forgetting that array indices start at 0, or writing the loop condition incorrectly (e.g., `i ≤ numStudents`), will lead to an **array index out of bounds** error. Accessing `scores[5]` is a serious error because it accesses memory that doesn't belong to the array.

4.4 数组与函数 / Arrays and Functions

中文

当你需要编写一个函数来处理数组时，比如打印数组或对数组排序，你需要了解数组是如何传递给函数的。

关键点：数组是按引用传递的。
这意味着当你把数组名作为参数传递给函数时，你传递的不是整个数组的副本，而是数组在内存中的**起始地址**。因此，函数内部对数组元素的任何修改，都会**直接改变**原始数组。

函数声明示例：

```
// 函数声明，用于打印数组
void printArray(int arr[], int size);

// 函数声明，用于修改数组（例如，将所有元素翻倍）
void doubleArray(int arr[], int size);
```

注意，在函数参数中，`int arr[]` 并不指定数组的大小。这是因为编译器只关心这是一个数组（实际上是一个指针）。因此，我们**必须**额外传递一个 `int size` 参数，告诉函数数组里有多少个元素。

示例：
```cpp

# include

using namespace std;

void doubleArray(int arr[], int size) {
for (int i = 0; i < size; i++) {
arr[i] = arr[i] * 2; // 直接修改原始数组
}
}
```

int main() {

int myArray[] = {1, 2, 3, 4, 5};

int size = 5;

```
doubleArray(myArray, size); // 传递数组和大小

// 打印结果，验证原始数组已被修改
```